

Interphone vidéo

Par Nasir Afzal

Sommaire

1. Introduction	3
Liste de composant /modules :	3
2. Description du système	4
2.1 schémas bloc :	4
2.3 Explication :	6
Schéma connections :	7
Model 3d :	8
Bloc 1 : communication espnow :	10
Informations générales sur ESP-NOW	10
Facteurs de performance	13
Codes :	14
Code d'envoi :	14
Code récepteur :	15
Adresse mac :	16
Définition	16
Pratique :	16
Distances en différents modes :	17
Bibliographie et webographie :	39

1. Introduction

Ce projet consiste à réaliser un interphone vidéo sans fil permettant de voir et de parler avec une personne qui sonne à la porte.

Le système utilise une caméra et un micro à l'extérieur, ainsi qu'un écran et un haut-parleur à l'intérieur, afin de transmettre à la fois l'image et le son entre les deux côtés de la porte.

La communication entre les deux modules est assurée par des microcontrôleurs ESP32, qui échangent les données via le protocole sans fil ESP-NOW.

Liste de composant /modules :

Seed studio esp32s3 sens :

https://wiki.seeedstudio.com/xiao_esp32s3_pin_multiplexing/

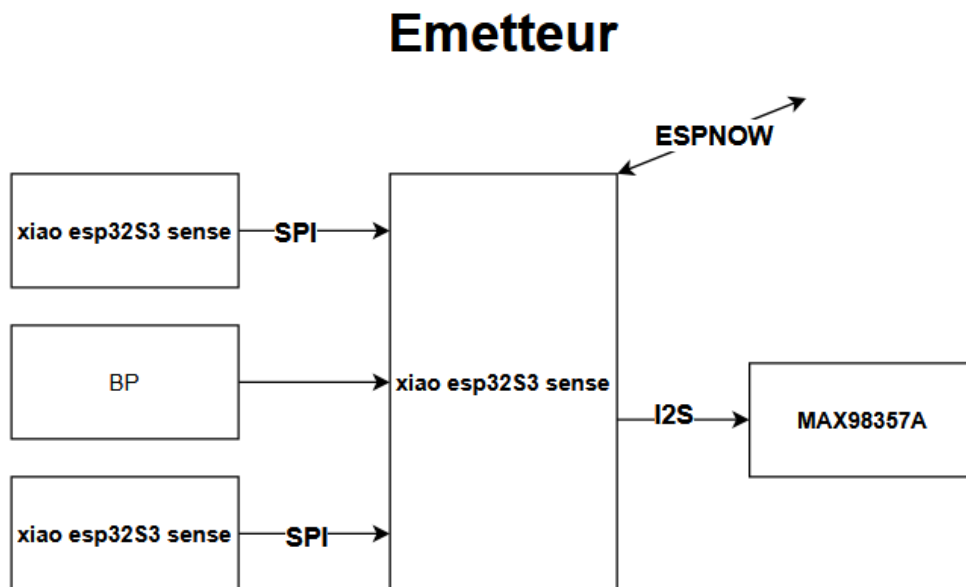
Ili9341 tft : <https://www.otronic.nl/fr/ecran-tactile-tft-28-pouces-320x240-ili9341-spi.html>

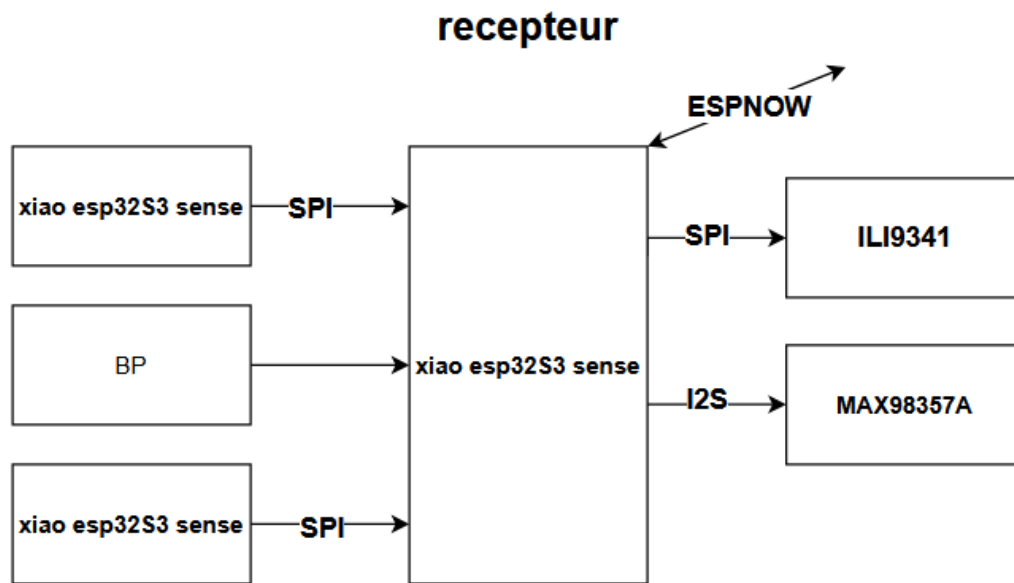
Max98357A : <https://learn.adafruit.com/adafruit-max98357-i2s-class-d-mono-amp/overview>

Bms + booster 5v : <https://www.adafruit.com/product/6106>

2. Description du système

2.1 schémas bloc :





2.3 Explication :

Camera : elle va enregistrer le flux vidéo live et l'envoyer au microcontrôleur. Le moyen de transmission est encore non décider.

Micro : il va enregistrer l'audio et l'envoyer au microcontrôleur.

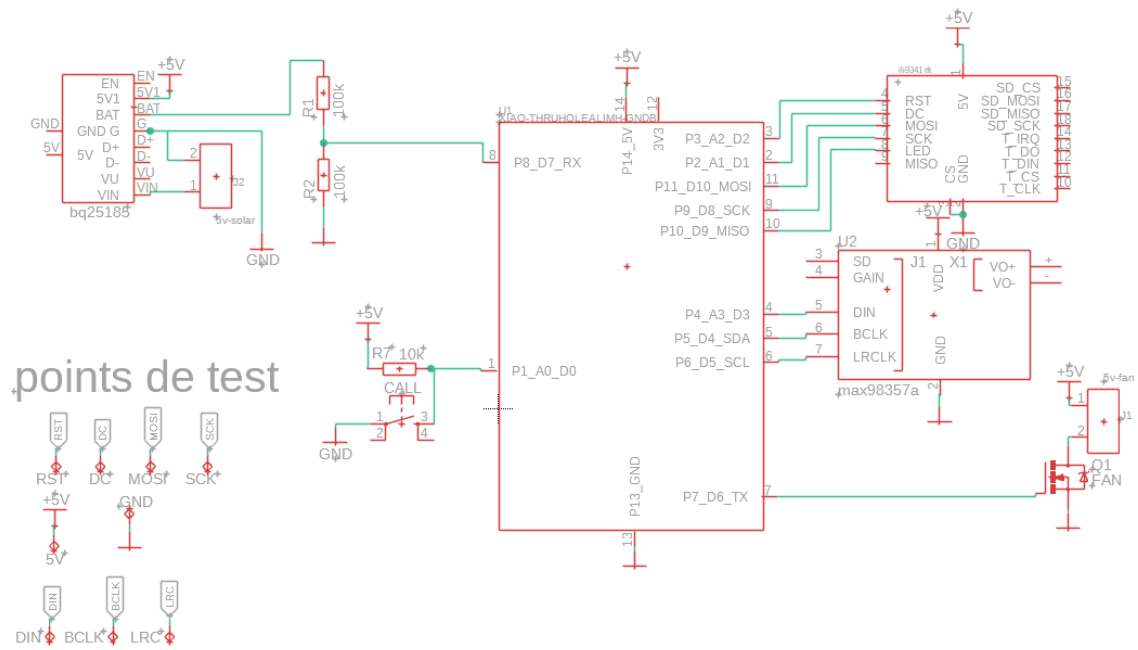
Microcontrôleur : il me permet de gérer les données reçues et de les traiter et envoyer vers le récepteur.

Ecran : c'est un écran qui va afficher l'image de la caméra.

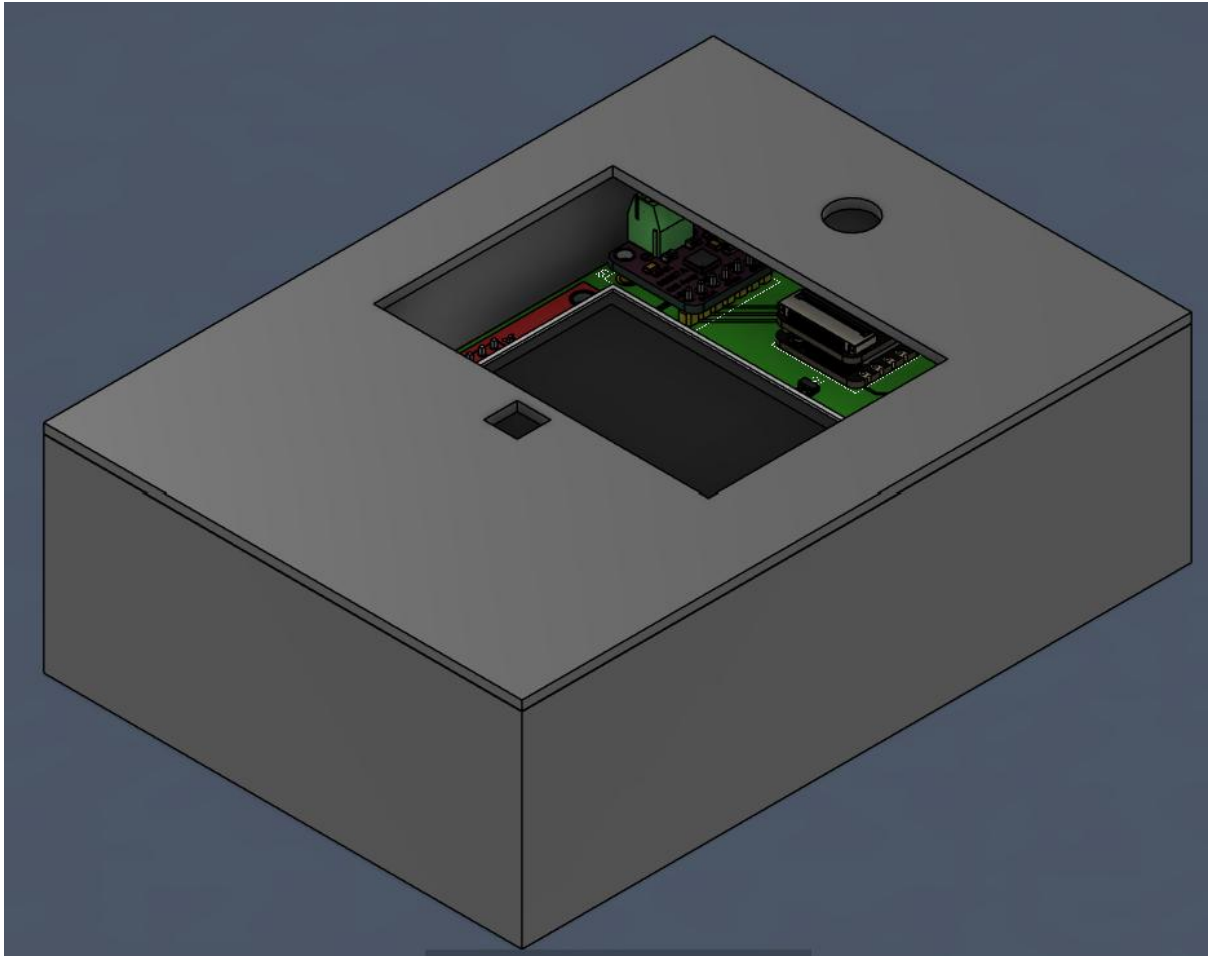
Haut-Parleur : Il va émettre le son reçu depuis le micro.

Microcontrôleur récepteur : il sera le récepteur qui va recevoir toutes les informations transmises par le transmetteur qui lui envoie le son et l'image.

Schéma connections :



Model 3d :



Bloc 1 : communication espnow :

ESP-NOW est un protocole de communication créé par Espressif pour la communication directe en sans-fil entre microcontrôleurs ESP32 et ESP8266.

Dans ce projet, il est utilisé pour transmettre les données entre le module extérieur et le module intérieur.

Informations générales sur ESP-NOW

- **Fréquence radio utilisée** : ESP-NOW fonctionne uniquement sur la fréquence de 2,4 GHz (même bande que le Wi-Fi et le Bluetooth).
- **Débit maximal** : Jusqu'à 1 Mbps en mode standard ; le mode Long Range (LR) réduit le débit à 256–512 Kbps pour améliorer la portée.
- **Taille maximale d'un paquet** : 250 octets (payload), soit la quantité maximale de données envoyée en un seul message. Pour de plus gros transferts, il faut fragmenter les données en plusieurs paquets.

Portée :

- En extérieur : 200 à 650 m en mode standard, jusqu'à 900 m en mode Long Range avec antenne adaptée
- En intérieur : typiquement 40 à 100 m, dépendant des obstacles et matériaux

Type de communication : Peer-to-peer direct (one-to-one), diffusion (broadcast, one-to-many, many-to-one), maillage possible

On fait qu'envoyer



One to many



Many to one



Et le plus important la communication double

Peer to peer.



Facteurs de performance

- Type et orientation d'antenne (PCB ou externe)
- Environnement (champ libre vs. Obstacles, murs)
- Fréquence et canal Wi-Fi

Codes :

Code d'envoi :

```
#include <WiFi.h>
#include <esp_now.h>

uint8_t receiverMAC[] = {0x24,0x6F,0x28,0xAA,0xBB,0xCC};
int n = 0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  if (esp_now_init() != ESP_OK) { Serial.println("ESP-NOW erreur"); return; }
  esp_now_peer_info_t p = {};
  memcpy(p.peer_addr, receiverMAC, 6);
  esp_now_add_peer(&p);
}

void loop() {
  esp_now_send(receiverMAC, (uint8_t*)&n, sizeof(n));
  Serial.print("Envoye: "); Serial.println(n);
  n++;
  delay(500);
}
```

Le code d'envoi initialise le Wi-Fi en mode station, configure ESP-NOW, enregistre l'adresse MAC du récepteur comme unique pair, puis envoie régulièrement un entier qui s'incrémente à chaque transmission

Code récepteur :

```
#include <WiFi.h>
#include <esp_now.h>

int n = 0;

void onReceive(const uint8_t *mac, const uint8_t *data, int len) {
    memcpy(&n, data, sizeof(n));
    Serial.print("Recu: "); Serial.println(n);
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    if (esp_now_init() != ESP_OK) { Serial.println("ESP-NOW erreur"); return; }
    esp_now_register_recv_cb(onReceive);
}

void loop() {
}
```

Le code de réception enregistre une fonction de rappel (callback) qui est appelée à chaque paquet reçu, copie la valeur reçue dans une variable et l'affiche sur le port série.

Adresse mac :

Définition

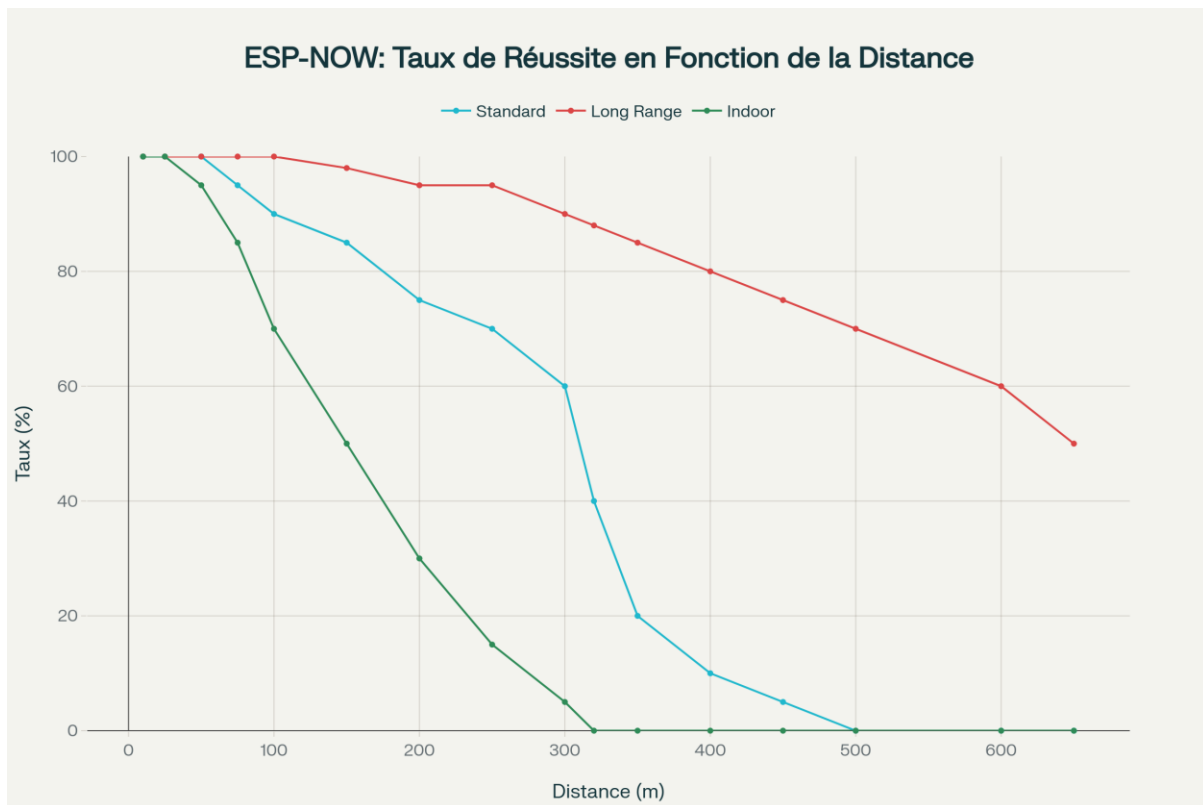
Une **adresse MAC** (Media Access Control) est un identifiant unique qui permet de reconnaître chaque appareil connecté à un réseau informatique.

Pratique :

Dans le cadre d'ESP-NOW, chaque ESP32 utilise son adresse MAC Wi-Fi pour envoyer et recevoir les paquets.

Dans ce projet, l'adresse MAC du module récepteur est enregistrée dans le programme de l'émetteur afin de s'assurer que les données audio et les messages de contrôle sont envoyés précisément vers l'ESP intérieur, et non vers un autre appareil Wi-Fi présent dans l'environnement.

Distances en différents modes :



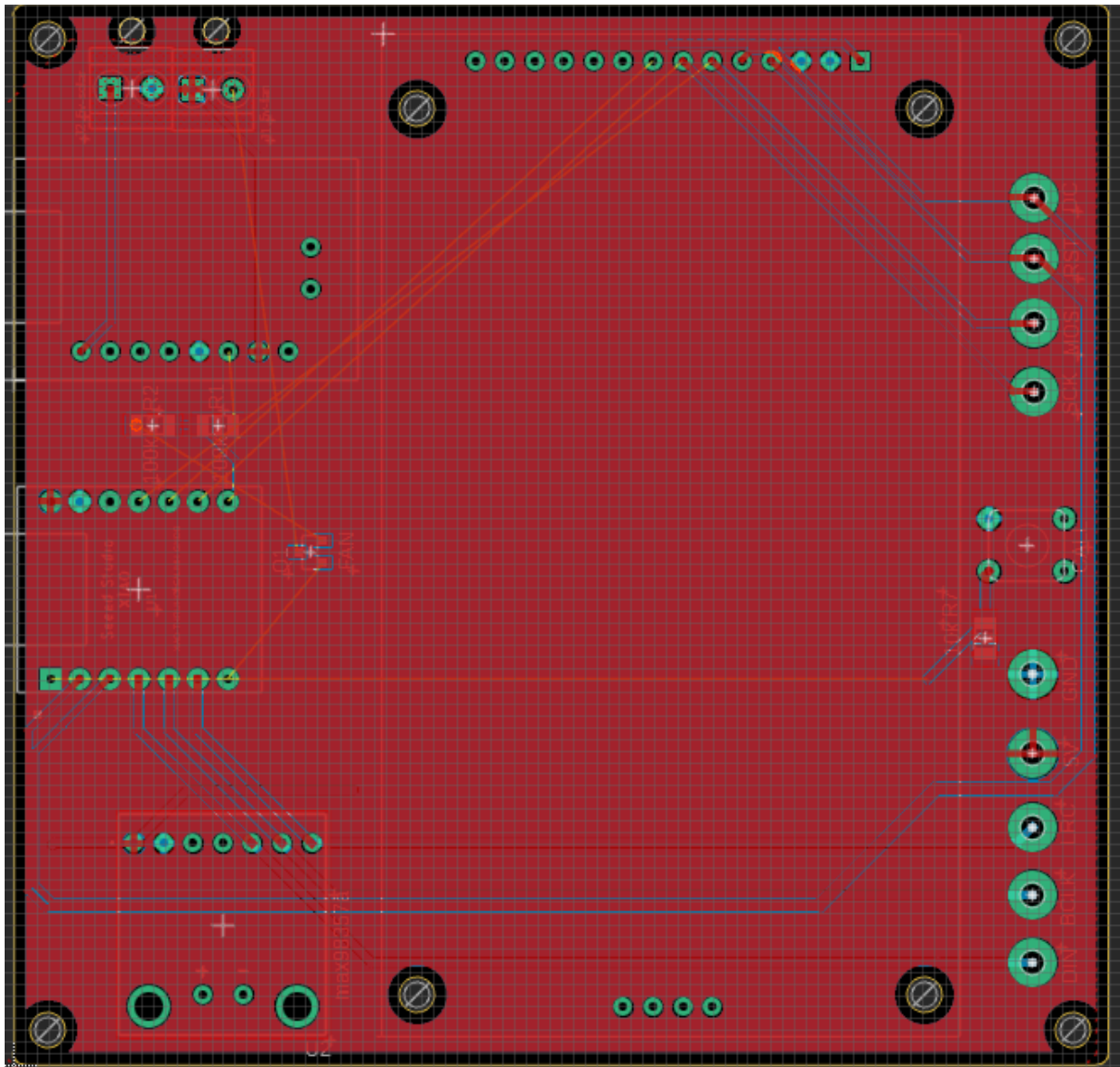
<https://www.e-tinkers.com/2020/05/esp-01-range-test-with-esp-now-protocol/>

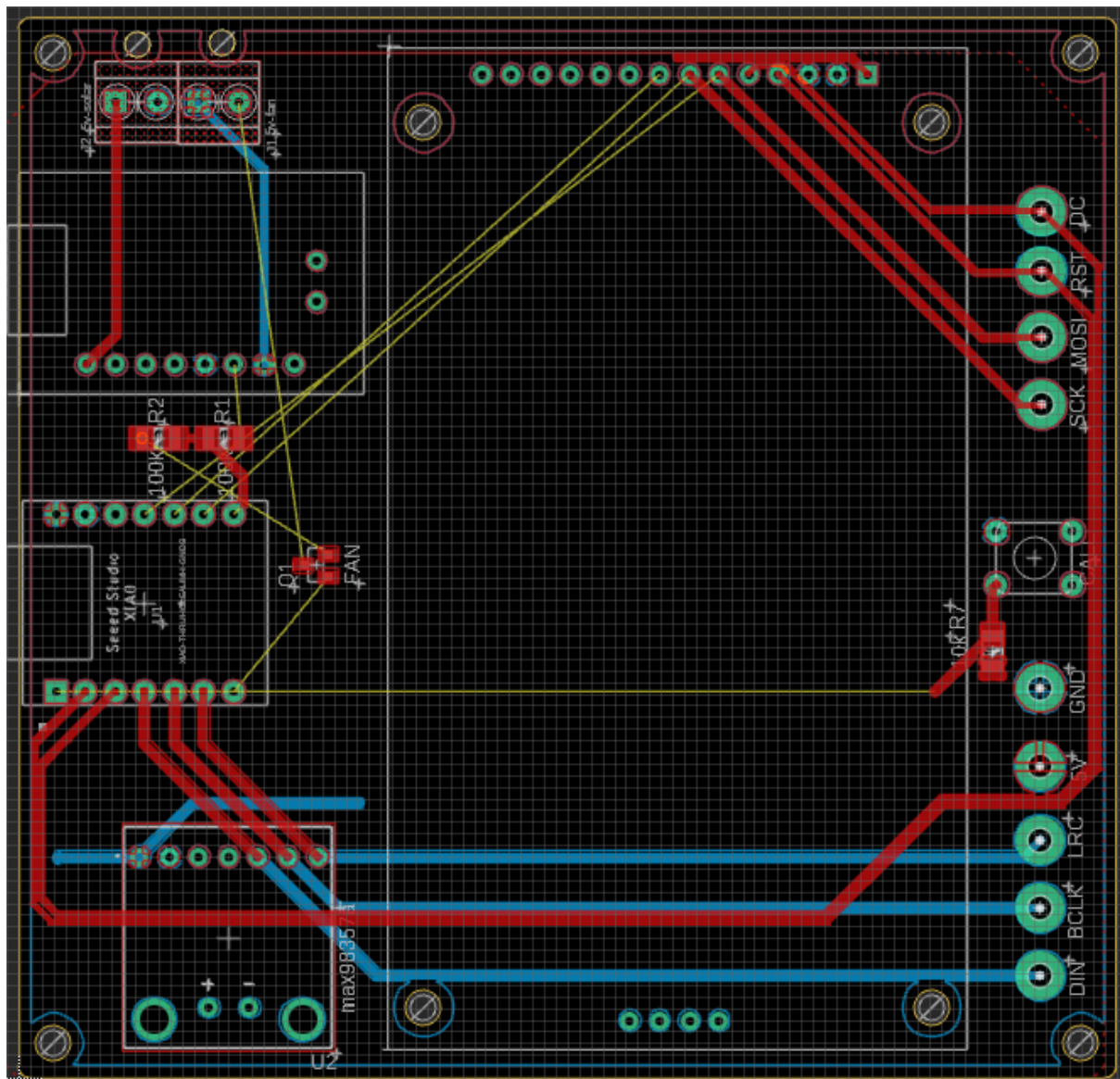
https://www.reddit.com/r/esp32/comments/o1xpzn/espnov_real_world_range_test_standard_and_long/

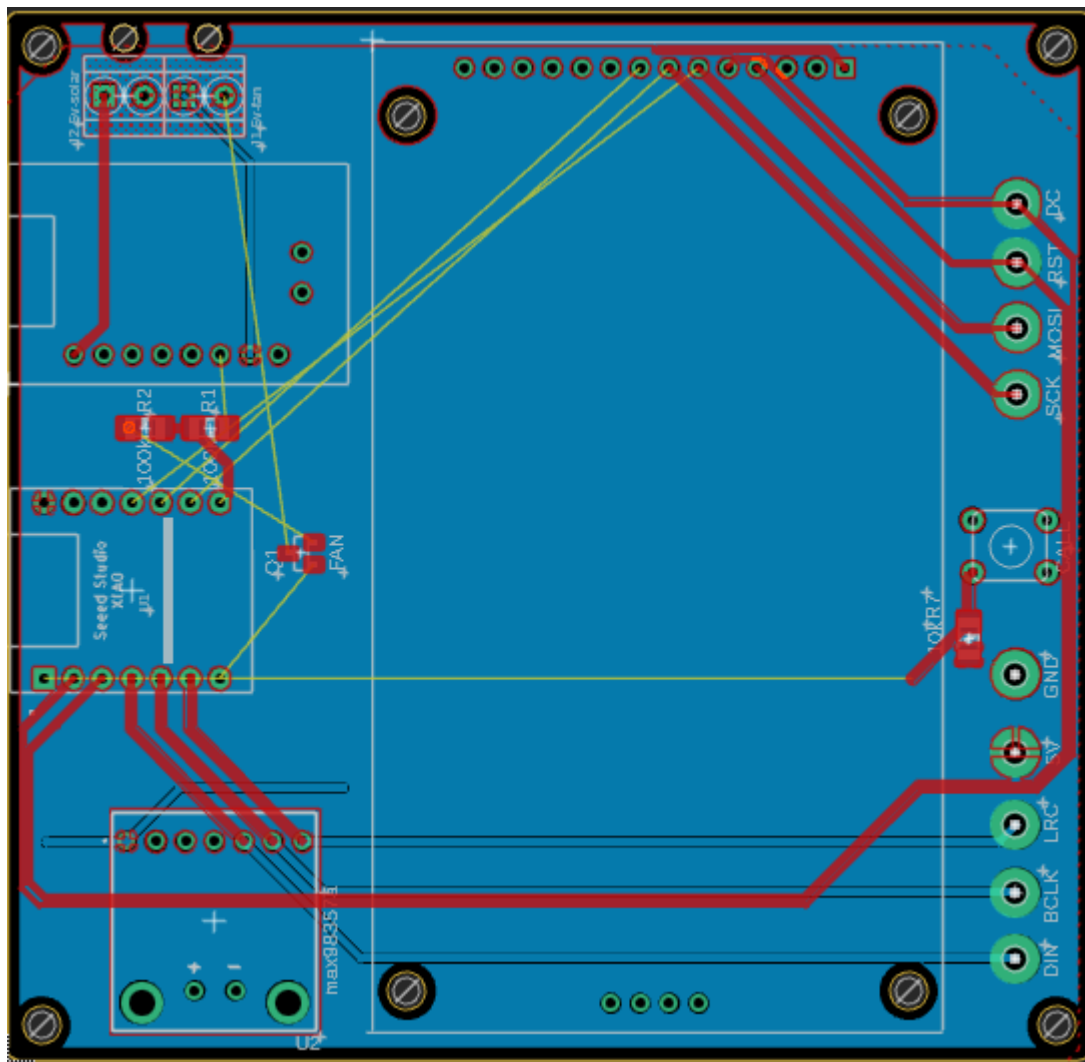
Ce graphique est basé sur des tests réels fait par moi et d'autres et permet de visualiser à partir de quelle distance les pertes de paquets deviennent importantes, surtout en intérieur.

Nous pouvons voir que l'environnement fermé et les signaux wifi, influence la distance de transmission.

PCB :







Le bus SPI (Serial Peripheral Interface)

Le bus SPI (Serial Peripheral Interface) est un protocole de communication série synchrone et full duplex utilisé pour échanger des données à grande vitesse entre un microcontrôleur maître et un ou plusieurs périphériques esclaves. Grâce à son débit élevé — souvent supérieur à 1 Mbit/s — il est particulièrement adapté aux applications nécessitant des transferts rapides, comme les écrans LCD.

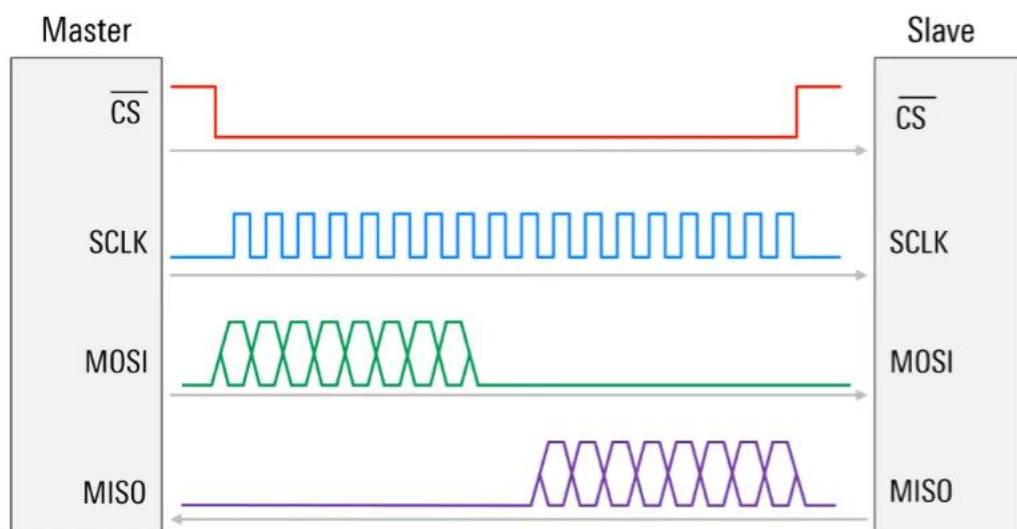
Les 4 pins importantes :

SCLK (Serial Clock) : signal d'horloge fourni par le maître, assurant la synchronisation des échanges.

MOSI (Master Out Slave In) : ligne de transmission des données du maître vers l'esclave.

MISO (Master In Slave Out) : ligne de transmission des données de l'esclave vers le maître.

CS (Chip Select): signal permettant de sélectionner l'esclave actif, généralement actif à l'état bas.



Source : <https://www.youtube.com/watch?v=0nVNwozXslc&t=119s>

Principe de fonctionnement

Une communication SPI commence lorsque le maître met la ligne CS à l'état bas, ce qui sélectionne l'esclave concerné et déclenche l'échange de données.

Les bits sont alors envoyés par le maître sur la ligne MOSI, cadencés par le signal SCLK, tandis que l'esclave peut simultanément transmettre des données vers le maître via MISO.

À la fin de la transmission, le maître remet CS à l'état haut, signalant la fin de l'échange.

Modes de fonctionnement

Le protocole SPI propose quatre modes de fonctionnement, définis à partir de deux paramètres :

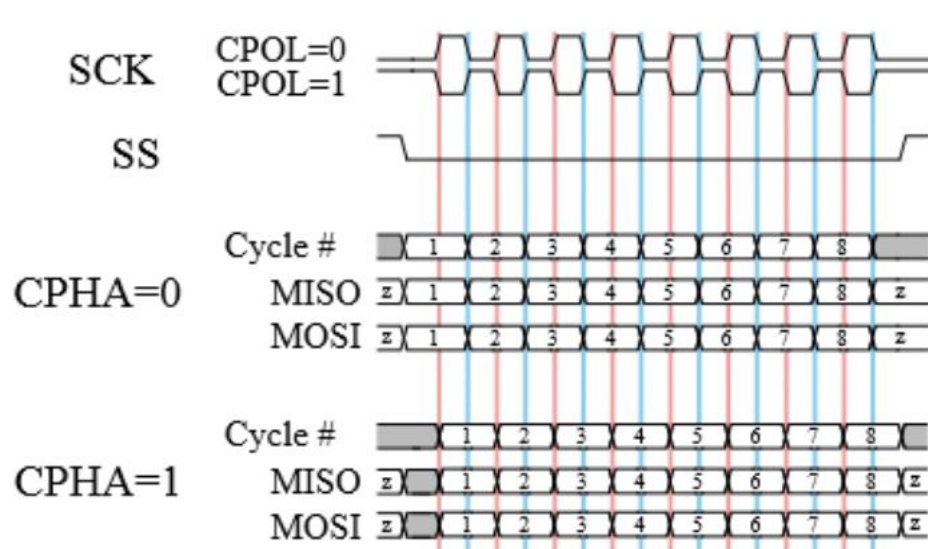
CPOL (Clock Polarity) : indique le niveau de repos de l'horloge (0 pour bas, 1 pour haut).

CPHA (Clock Phase) : précise le front de l'horloge utilisé pour l'échantillonnage des données.

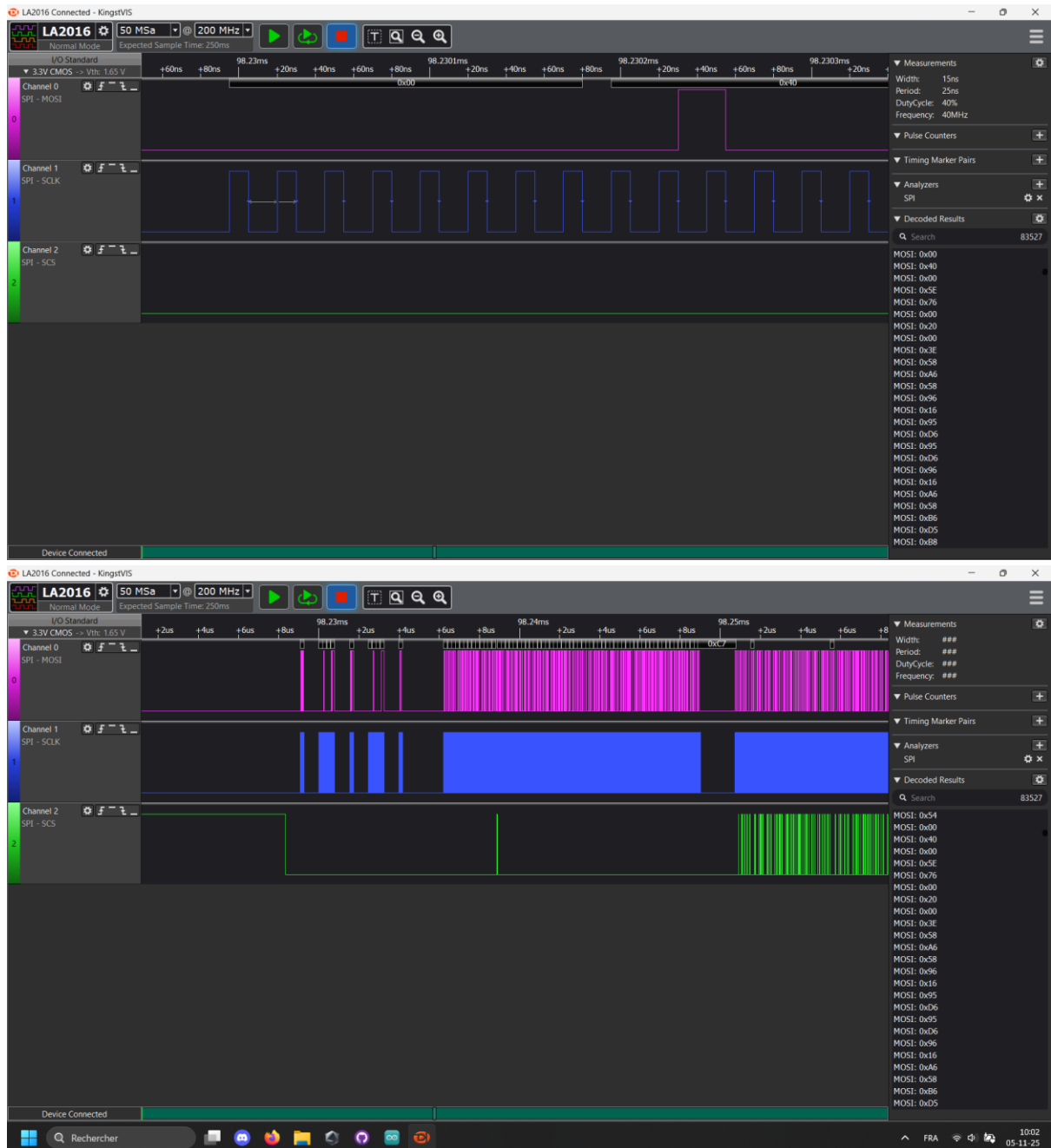
Dans ce cas précis, le mode 0 est utilisé, correspondant à CPOL = 0 et CPHA = 0.

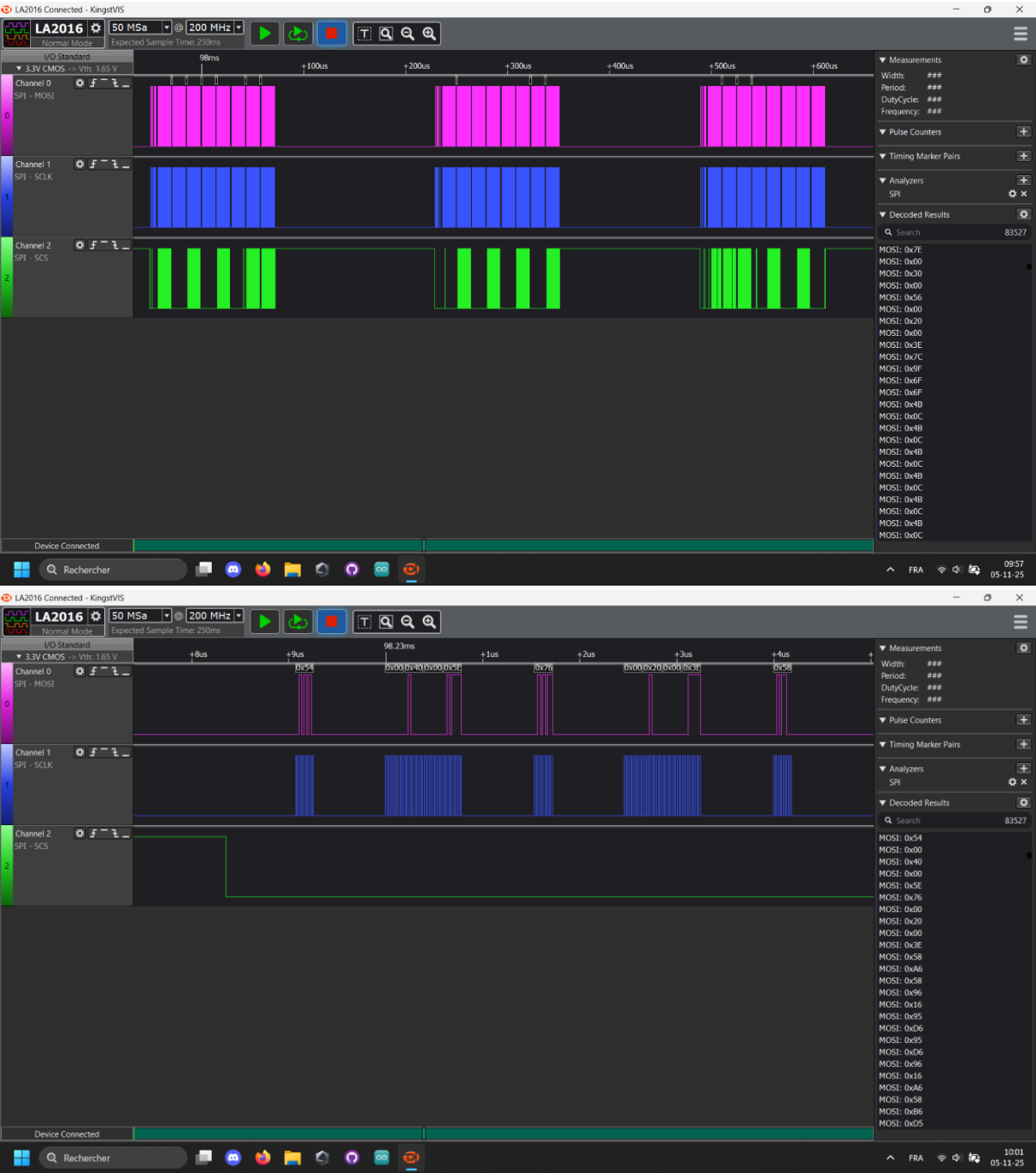
Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Source : <https://www.youtube.com/watch?v=0nVNwozXslc&t=119s>



Mesures :





Ces mesures sont réalisées à l'aide d'un analyseur de signaux, ce qui permet d'obtenir des signaux bien définis. En revanche, avec l'oscilloscope dont nous disposons, limité à une bande passante de 50 MHz, les signaux ne sont pas observés sous une forme carrée parfaite, mais apparaissent plutôt comme un sinus.

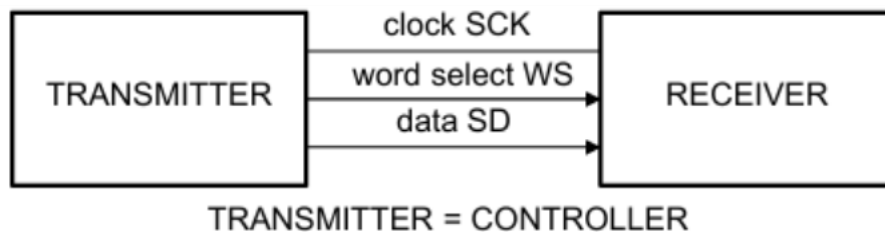
En effet, les harmoniques qui composent un signal carré sont fortement atténuées par l'effet du filtre passe-bas dû à la sonde et à la bande passante limitée de l'oscilloscope.

I2S :

Le bus I²S (Inter-IC Sound) est un protocole de communication série synchrone dédié à la transmission de données audio numériques entre un microcontrôleur et des périphériques tels qu'un DAC ou un ADC. Conçu spécifiquement pour l'audio.

Le bus I²S utilise en général trois lignes principales:

- BCLK ou SCK (Bit Clock / Serial Clock)
Il s'agit de l'horloge générée par le maître pour cadencer la transmission des bits. Chaque période d'horloge correspond à l'envoi d'un bit de donnée.
- LRCLK ou WS (Left Right Clock / Word Select)
Cette ligne indique le canal audio en cours de transmission :
 - niveau bas → canal gauche
 - niveau haut → canal droitElle change d'état à chaque nouvel échantillon audio.
- SD ou SDATA (Serial Data)
Ligne sur laquelle sont transmises les données audio numériques, sous forme de suites de bits représentant les échantillons.



Principe de fonctionnement (mode stéréo)

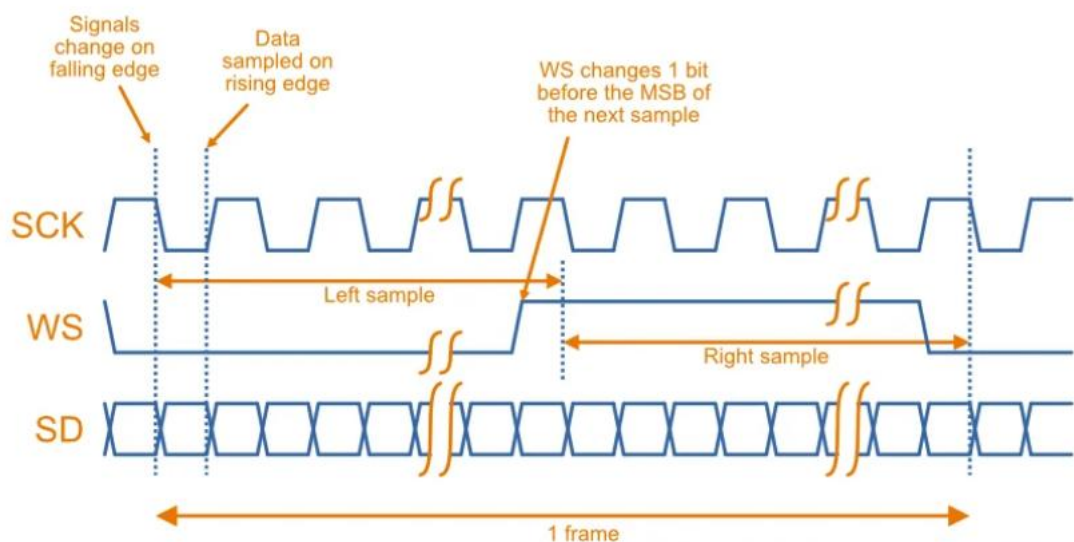
Dans une communication I²S, le maître fournit à la fois l'horloge BCLK et le signal WS. Les données audio sont transmises en continu sur la ligne SD, synchronisées par BCLK.

Lorsque WS est à l'état bas, les données correspondent au canal gauche.

Lorsque WS est à l'état haut, elles correspondent au canal droit.

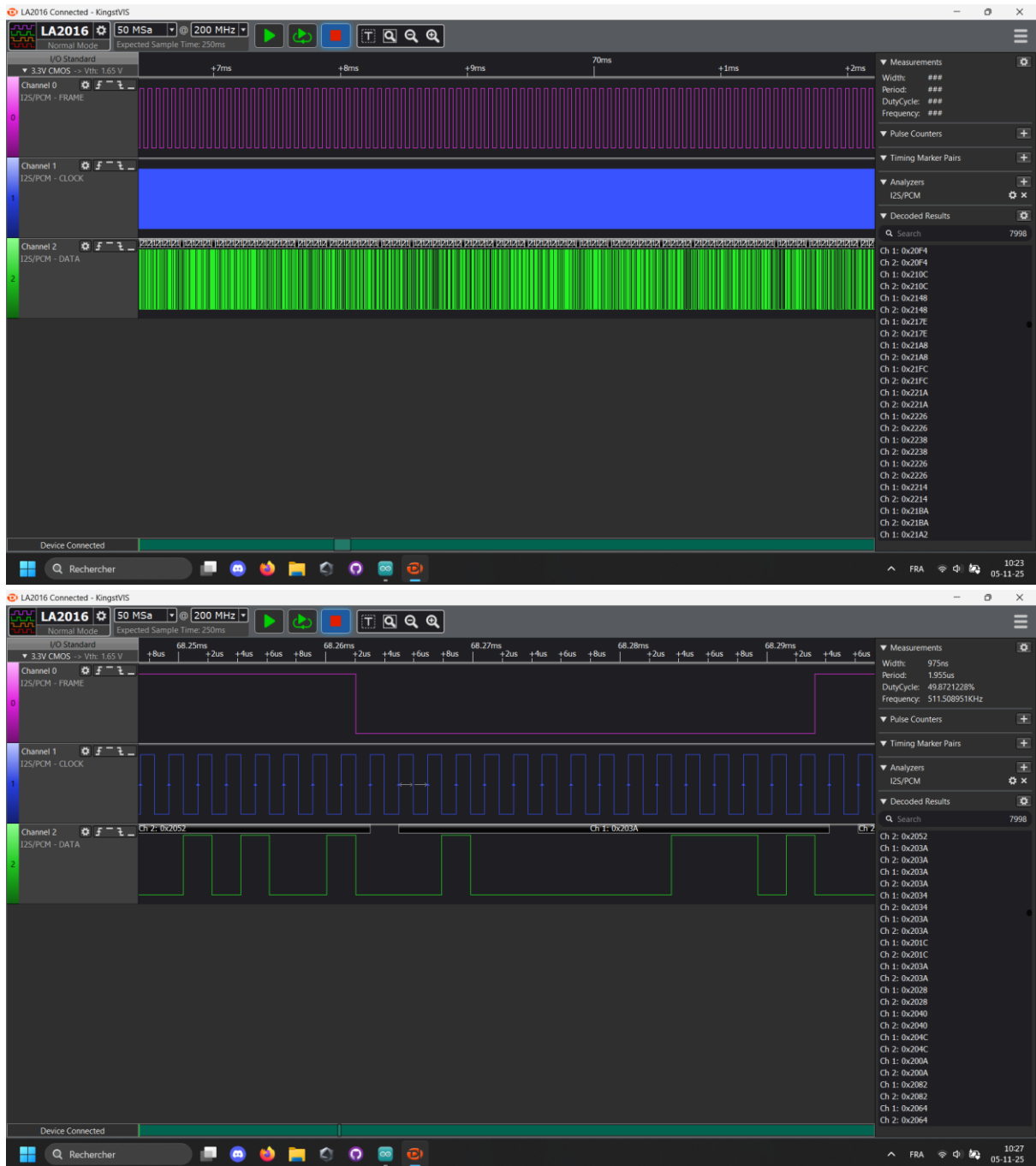
Le protocole I²S repose sur un timing fixe et normalisé : les données changent sur le front descendant de l'horloge BCLK et sont échantillonnées sur le front montant.

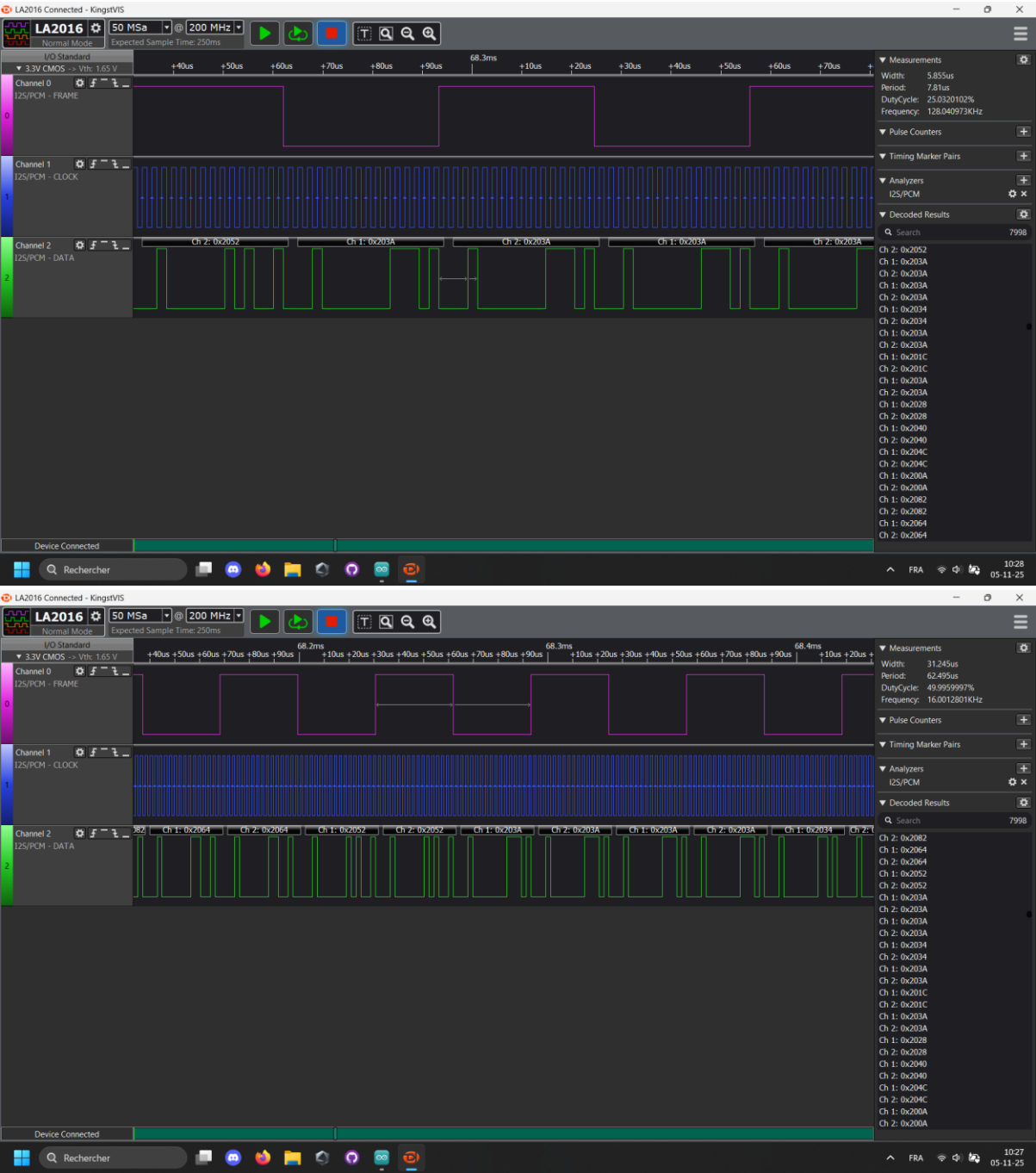
De plus, les données ne sont pas envoyées immédiatement après le changement d'état de WS. Le protocole impose un décalage d'un cycle d'horloge : après la transition de WS, un front d'horloge supplémentaire est attendu avant la transmission du bit de poids fort (MSB) du nouvel échantillon. Ce décalage assure une synchronisation fiable entre la sélection du canal et les données transmises.



<https://blog.mbedded.ninja/electronics/communication-protocols/i2s-communication-protocol/>

mesures :





Code test :

Nasir Afzal

```
/******

* Nom      : Nasir Afzal

* Matériel : Xiao ESP32-S3 Sense

* Afficheur : TFT ILI9341

* Audio    : MAX98357A (I2S)

* Description:

* - Lecture d'une mélodie via I2S

* - Affichage de la note jouée sur écran TFT

* - Déclenchement par bouton

*****/

#include <Arduino.h>

#include <TFT_eSPI.h>

#include <driver/i2s.h>

/* =====

OBJETS GLOBAUX

===== */

// Objet TFT (configuré via User_Setup.h de TFT_eSPI)
TFT_eSPI tft = TFT_eSPI();

/* =====

CONFIGURATION DES BROCHES

===== */

// Bouton de déclenchement
const int TRIGGER_PIN = 43; // GPIO43 (D6) - bouton externe

// Sorties digitales
const int OUT_PIN1 = 1; // GPIO1
const int OUT_PIN2 = 8; // GPIO8

/* =====

CONFIGURATION I2S (MAX98357A)

===== */

#define I2S_DOUT 4 // Données audio
#define I2S_BCLK 5 // Bit Clock
```

Nasir Afzal

```
#define I2S_LRC 6      // Left/Right Clock

#define SAMPLE_RATE 44100 // Fréquence d'échantillonnage audio
#define VOLUME 0.5f      // Volume (0.0 à 1.0)

/* =====
STRUCTURE DES NOTES
===== */

// Structure représentant une note musicale
struct Note {
    const char* name; // Nom affiché
    float freq;      // Fréquence en Hz
    int duration;    // Durée en millisecondes
};

// Mélodie à jouer
Note melody[] = {
    {"C4", 261.63, 400},
    {"D4", 293.66, 400},
    {"E4", 329.63, 400},
    {"F4", 349.23, 400},
    {"G4", 392.00, 400},
    {"A4", 440.00, 400},
    {"B4", 493.88, 400},
    {"C5", 523.25, 600},
};

/* =====
INITIALISATION I2S
===== */

void i2s_install() {

    // Configuration du périphérique I2S
    const i2s_config_t i2s_config = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
        .sample_rate = SAMPLE_RATE,
        .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
```

```
.channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,

.communication_format = I2S_COMM_FORMAT_STAND_I2S,

.intr_alloc_flags = 0,

.dma_buf_count = 8,

.dma_buf_len = 64,

.use_apll = false,

.tx_desc_auto_clear = true,

.fixed_mclk = 0
};

// Attribution des broches I2S
const i2s_pin_config_t pin_config = {

.bck_io_num = I2S_BCLK,

.ws_io_num = I2S_LRC,

.data_out_num = I2S_DOUT,

.data_in_num = I2S_PIN_NO_CHANGE
};

// Installation et démarrage du driver I2S
i2s_driver_install(I2S_NUM_0, &i2s_config, 0, NULL);

i2s_set_pin(I2S_NUM_0, &pin_config);

i2s_set_sample_rates(I2S_NUM_0, SAMPLE_RATE);
}

/* =====
GÉNÉRATION D'UN SON
===== */

void playTone(float freq, int duration_ms){

// Nombre total d'échantillons à produire
const int samples = SAMPLE_RATE * duration_ms / 1000;

// Amplitude maximale (16 bits)
const float amplitude = 32767 * VOLUME;

float phase = 0.0f;

float phase_increment = 2 * PI * freq / SAMPLE_RATE;
```


Nasir Afzal

```
int16_t buffer[256];

int samples_sent = 0;

// Envoi des échantillons par blocs
while (samples_sent < samples) {
    for (int i = 0; i < 256; i++) {
        buffer[i] = (int16_t)(sin(phase) * amplitude);
        phase += phase_increment;
        if (phase >= 2 * PI) phase -= 2 * PI;
    }

    size_t bytes_written;
    i2s_write(I2S_NUM_0, buffer, sizeof(buffer),
        &bytes_written, portMAX_DELAY);

    samples_sent += 256;
}

/* =====
AFFICHAGE DE LA NOTE SUR LE TFT
===== */

void showNoteOnTFT(const char* note) {

    tft.fillScreen(TFT_BLACK);
    tft.setTextColor(TFT_GREEN, TFT_BLACK);
    tft.setTextDatum(MC_DATUM);

    // Police large pour lecture claire
    tft.setFreeFont(&FreeSansBold24pt7b);

    // Affichage centré
    tft.drawString(note,
        tft.width() / 2,
        tft.height() / 2);
}

/* =====
```

Nasir Afzal

```
SETUP

===== */

void setup() {

    Serial.begin(115200);

    // Configuration des E/S
    pinMode(TRIGGER_PIN, INPUT); // Résistance externe
    pinMode(OUT_PIN1, OUTPUT);
    pinMode(OUT_PIN2, OUTPUT);

    digitalWrite(OUT_PIN1, LOW);
    digitalWrite(OUT_PIN2, LOW);

    // Initialisation de l'écran TFT
    tft.init();
    tft.setRotation(1);
    tft.fillScreen(TFT_BLACK);
    tft.setTextSize(2);
    tft.setTextDatum(MC_DATUM);
    tft.setTextColor(TFT_WHITE, TFT_BLACK);

    tft.drawString("MAX98357A Tone Demo",
        tft.width() / 2,
        tft.height() / 2 - 40);

    delay(1500);

    // Démarrage de l'I2S
    i2s_install();
}

/* =====

LOOP PRINCIPALE

===== */

void loop() {
```

Nasir Afzal

```
// Bouton appuyé (niveau bas)

if (digitalRead(TRIGGER_PIN) == LOW) {

    unsigned long start = millis();

    // Activation des sorties

    digitalWrite(OUT_PIN1, HIGH);
    digitalWrite(OUT_PIN2, HIGH);

    // Lecture pendant 2 secondes

    while (millis() - start < 2000) {

        for (auto &n : melody) {

            if (millis() - start >= 2000) break;

            Serial.println(n.name);
            showNoteOnTFT(n.name);
            playTone(n.freq, n.duration);

            delay(50); // pause entre notes
        }
    }

    // Désactivation des sorties

    digitalWrite(OUT_PIN1, LOW);
    digitalWrite(OUT_PIN2, LOW);

    delay(200); // anti-rebond
}
}
```

Bloc 2 : compression d'images en jpg :

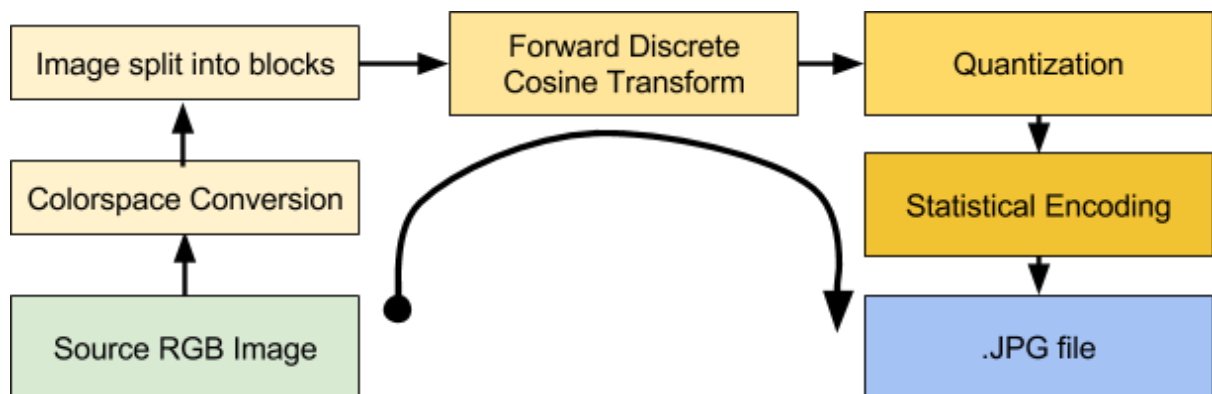
Qu'est-ce que la compression JPEG ?

JPEG (*Joint Photographic Experts Group*) est un format de compression avec perte conçu pour :

- réduire fortement la taille des images,
- tout en conservant une qualité visuelle acceptable pour l'œil humain.

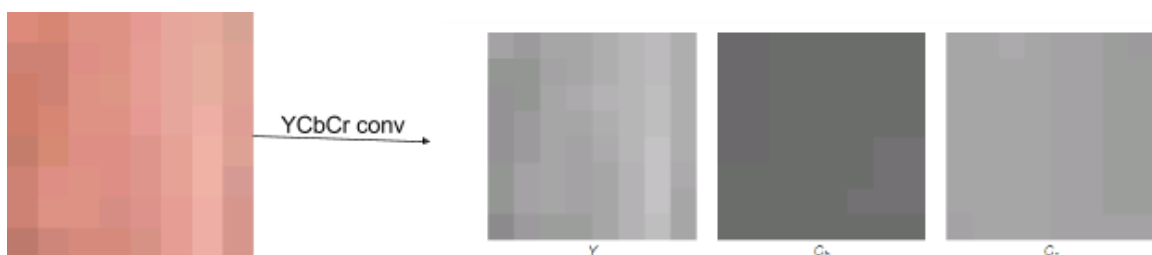
Étape 1 : Conversion de l'espace couleur (RGB → YCbCr)

- RGB = Rouge, Vert, Bleu
- JPEG convertit en YCbCr :
 - Y : luminance (luminosité)
 - Cb / Cr : chrominance (couleur)



Sous-échantillonnage des couleurs

- La luminance est conservée partout
- Les couleurs sont stockées à plus basse résolution

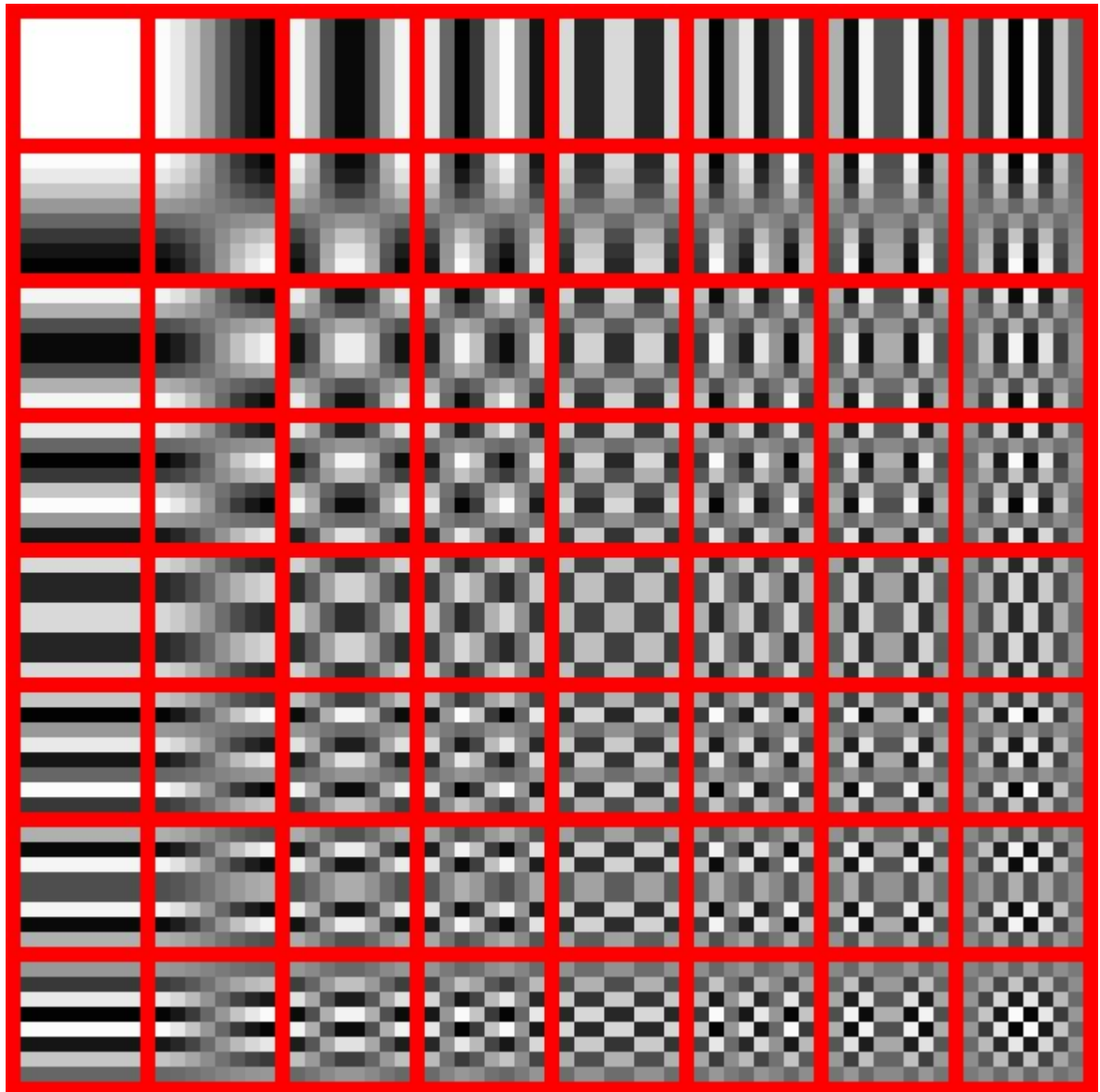


Découpage en blocs 8×8

L'image est découpée en blocs de 8×8 pixels.

Pourquoi 8×8 ?

- Bon compromis entre qualité et efficacité
- Compatible avec le matériel de l'époque (années 90)



Transformation en cosinus discrète (DCT)

La DCT transforme les pixels du bloc en fréquences spatiales :

- Basses fréquences → zones lisses
- Hautes fréquences → détails fins, contours, bruit

Formule simplifiée :

$$DCT(u, v) = \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos(\dots) \cos(\dots)$$



(a) Uncompressed image



(b) JPEG image



(c) zoomed-in view of JPEG image

Source : <https://www.freecodecamp.org/news/how-jpg-works-a4dbd2316f35/>

Bibliographie et webographie :

<https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>

<https://www.e-tinkers.com/2020/05/esp-01-range-test-with-esp-now-protocol/>

https://www.reddit.com/r/esp32/comments/o1xpzn/espnow_real_world_range_test_standard_and_long/

<https://www.espressif.com/en/solutions/low-power-solutions/esp-now>